Window Functions and CTEs

Window functions perform calculations across a set of rows **related to the current row**, without collapsing results into a single row like `GROUP BY`. **CTEs (Common Table Expressions)** use the `WITH` clause to define temporary named result sets.

> **Note:** Window functions and CTEs require **MySQL 8.0+**.

Module Outcome:

- Rank or running-total **without** collapsing rows like GROUP BY
- Name a CTE with `WITH` so a nested mess becomes a story
- Remember this needs MySQL 8 — lab PCs on 5.7 will lie to you

Window Functions Overview

Unlike aggregate functions with `GROUP BY`, window functions keep all rows and add a computed column.

```
SELECT
    column1,
    aggregate_function(column2) OVER (PARTITION BY column3 ORDER BY column4) AS alias
FROM
    table_name;
```

Clause	Description
`OVER()`	Defines the window (row set) for the calculation
`PARTITION BY`	Divides rows into groups (like GROUP BY, but keeps all rows)
`ORDER BY`	Sorts rows within each partition
`ROWS BETWEEN`	Defines which rows to include in the frame (optional)

ROW_NUMBER()

Assigns a unique sequential integer to each row within a partition, starting at 1.

Syntax:

```
ROW_NUMBER() OVER (PARTITION BY column ORDER BY column) AS alias
```

Example: Rank products by price within each category.

```
SELECT
    name,
    category,
    price,
    ROW_NUMBER() OVER (PARTITION BY category ORDER BY price DESC) AS price_rank
```

```
FROM
    product;
```

Output:

name	category	price	price_rank
Smart TV	Gadgets	40000	1
Smart Watch	Gadgets	17000	2
Smart Cam	Gadgets	2600	3
Black Jeans	clothing	750	1
Blue Shirt	clothing	750	2
Blue Jeans	clothing	800	3

Example 2: Get the top-rated product in each category.

```
SELECT name, category, rating
FROM (
    SELECT
        name,
        category,
        rating,
        ROW_NUMBER() OVER (PARTITION BY category ORDER BY rating DESC) AS rn
    FROM product
) ranked
WHERE rn = 1;
```

RANK()

Assigns a rank to each row. Rows with equal values receive the same rank, and the next rank is skipped.

Syntax:

```
RANK() OVER (PARTITION BY column ORDER BY column) AS alias
```

Example: Rank players by score. Tied scores get the same rank; next rank is skipped.

```
SELECT
    name,
    score,
    RANK() OVER (ORDER BY score DESC) AS player_rank
FROM
    player_match_details;
```

Output:

name	score	player_rank
Ram	75	1
Lokesh	75	1
Joseph	68	3
David	62	4

Note: Ranks 1, 1, **3** (rank 2 is skipped because two players tied for 1st).

DENSE_RANK()

Like `RANK()`, but does **not** skip ranks after ties.

Syntax:

```
DENSE_RANK() OVER (PARTITION BY column ORDER BY column) AS alias
```

Example: Same data as RANK example.

```
SELECT
    name,
    score,
    DENSE_RANK() OVER (ORDER BY score DESC) AS dense_rank
FROM
    player_match_details;
```

Output:

name	score	dense_rank
Ram	75	1
Lokesh	75	1
Joseph	68	2
David	62	3

Function	Ties	Skips ranks?
`ROW_NUMBER()`	Different numbers for ties (arbitrary order)	No
`RANK()`	Same rank for ties	Yes
`DENSE_RANK()`	Same rank for ties	No

LIMIT vs TOP

MySQL uses `LIMIT` for pagination and top-N queries (covered in Querying with SQL).

MySQL:

```
SELECT * FROM product ORDER BY rating DESC LIMIT 5;
```

SQL Server uses `TOP` instead:

```
-- SQL Server syntax (not MySQL)
SELECT TOP 5 * FROM product ORDER BY rating DESC;
```

> **Note:** This course uses MySQL — always use `LIMIT`, not `TOP`.

Common Table Expressions (CTEs)

A CTE defines a temporary named result set using the `WITH` clause. It improves readability of complex queries.

Syntax:

```
WITH cte_name AS (
    SELECT ...
    FROM ...
    WHERE ...
)
SELECT *
FROM cte_name;
```

Example 1: Get products priced above the average.

```
WITH avg_price AS (
    SELECT AVG(price) AS avg_val FROM product
)
SELECT
    p.name,
    p.price,
    a.avg_val
FROM
    product p, avg_price a
WHERE
    p.price > a.avg_val;
```

Example 2: Get each brand's average rating and list products above their brand average.

```
WITH brand_avg AS (
    SELECT
        brand,
        AVG(rating) AS avg_rating
    FROM product
    GROUP BY brand
)
SELECT
```

```
    p.name,
    p.brand,
    p.rating,
    b.avg_rating
FROM
    product p
    INNER JOIN brand_avg b ON p.brand = b.brand
WHERE
    p.rating > b.avg_rating;
```

Multiple CTEs

You can define multiple CTEs separated by commas.

```
WITH
    high_rated AS (
        SELECT * FROM product WHERE rating > 4.5
    ),
    expensive AS (
        SELECT * FROM product WHERE price > 5000
    )
SELECT name, brand, rating, price
FROM high_rated
WHERE name IN (SELECT name FROM expensive);
```

Recursive CTEs

Recursive CTEs reference themselves — useful for hierarchical data (org charts, category trees).

Syntax:

```
WITH RECURSIVE cte_name AS (
    -- Anchor member (base case)
    SELECT ... FROM ... WHERE ...
    UNION ALL
    -- Recursive member
    SELECT ... FROM cte_name WHERE ...
)
SELECT * FROM cte_name;
```

Example: Employee hierarchy (manager → reports).

```
WITH RECURSIVE emp_hierarchy AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employee
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.id, e.name, e.manager_id, h.level + 1
    FROM employee e
    INNER JOIN emp_hierarchy h ON e.manager_id = h.id
)
SELECT id, name, level
```

```
FROM emp_hierarchy
ORDER BY level, name;
```

Note: Recursive CTEs are supported in MySQL 8.0+. Use them for org charts, bill of materials, and folder structures.

CTE vs Subquery

Feature	CTE (`WITH`)	Subquery
Readability	Better for complex queries	Can become nested and hard to read
Reusability	Can reference CTE multiple times	Must repeat or nest
Recursion	Supports `WITH RECURSIVE`	No native recursion
Performance	Similar; optimizer treats them similarly in MySQL 8+	Similar

Note: Use CTEs when a query has multiple logical steps. Use subqueries for simple one-off filters.